Binary Tree Node Counting

Modified version : complete project documentation

Contents

- 1. Problem Statement
- 2. Objective
- 3. Introduction
- 4. Suggested Modifications
- 5. Algorithm in Detail
- 6. Flowchart
- 7. Expected Output
- 8. Comparison of Recursive and Iterative Methods
- 9. Program (C++ Source Code)
- 10. Explanation of the Code
- 11. Working of the Program
- 12. Test Cases
- 13. Time and Space Complexity
- 14. Advantages and Limitations
- 15. Applications
- 16. Viva Questions and Answers
- 17. Conclusion
- 18. References

1. Problem Statement

Write a C++ program to create a binary tree and count the total number of nodes, leaf nodes, internal nodes, nodes having exactly one child, and nodes having exactly two children. The program should also display the values of all leaf nodes instead of displaying only their count. Additionally, implement both recursive and iterative methods to count the total number of nodes in the binary tree and compare their results, time complexity, and space requirements. The program should display all the calculated results clearly.

Item	Details
Input	A binary tree built in the program by linking nodes.
Output	Leaf values; counts of total, leaf, internal, one-child and two-child nodes; recursive and iterative totals; comparison message.
Methods	Recursive preorder traversal; iterative level-order traversal using a queue (and a stack as an alternative).

Item	Details
Language	C++ (iostream, queue, stack).

2. Objective

The main objective of this modified program is to understand binary tree structures and analyze different types of nodes using C++. The program extends the previous binary tree node-counting implementation by adding two-child node counting, displaying leaf node values, and comparing recursive and iterative counting methods.

Specific objectives

- To create a binary tree using C++.
- To count the total number of nodes.
- To count leaf nodes and display their values.
- To count internal nodes.
- To count nodes having exactly one child.
- To count nodes having exactly two children.
- To implement recursive node counting.
- To implement iterative node counting using a queue or stack.
- To compare recursive and iterative approaches.
- To understand the time and space complexity of both methods.

3. Introduction

A binary tree is a non-linear data structure in which each node can have a maximum of two children, called the left child and the right child. The first node of the tree is called the root node. Each node contains data and pointers that connect it to its children.

Binary trees are commonly used in searching algorithms, sorting techniques, expression evaluation, and hierarchical data representation. Counting different types of nodes helps us understand the structure and properties of a binary tree.

In the previous version of the program, the main operations were counting the total number of nodes, leaf nodes, internal nodes, and nodes having exactly one child. In the modified version, two additional features are introduced: counting nodes having exactly two children and displaying the values of leaf nodes.

Another important modification is the comparison between recursive and iterative counting methods. The recursive method uses function calls to traverse the tree, whereas the iterative method uses a queue or stack to visit nodes without recursive function calls. Both methods can calculate the total number of nodes, but their implementation and auxiliary space requirements differ.

Terminology

Term	Meaning
Root	Topmost node with no parent.
Leaf node	Node with no children (both pointers are NULL).
Internal node	Node with at least one child.

Term	Meaning
Degree of a node	Number of children of the node (0, 1 or 2).
Height (h)	Number of edges on the longest path from the root to a leaf.
Width (w)	Largest number of nodes on any one level.
Subtree	A node together with all its descendants.

4. Suggested Modifications

4.1 Count nodes having exactly two children

A node having exactly two children is a node whose left and right child pointers are both non-NULL.

```
if (root->left != nullptr &&
    root->right != nullptr) {
    twoChildNodes++;
}
```

For example, if node 1 has children 2 and 3, node 1 is counted as a node having exactly two children. This modification helps identify nodes with two branches in the binary tree.

4.2 Display the leaf nodes

Previously, the program displayed only the total number of leaf nodes. The modified program also displays the values stored in these nodes. A leaf node is a node that has neither a left child nor a right child.

```
if (root->left == nullptr &&
    root->right == nullptr) {
    leafNodes++;
    cout << root->data << " ";
}
```

Whenever a leaf node is identified, its value is displayed and the leaf node counter is increased.

4.3 Compare recursive and iterative counting methods

The modified program uses two approaches to count the total number of nodes.

- **Recursive method:** the function calls itself to process the left and right subtrees. It stops when it reaches a NULL pointer.
- **Iterative method:** the function uses a queue or stack to store nodes that must be visited. It processes the stored nodes one by one until all nodes have been counted.

Both methods produce the same total count when implemented correctly. However, they differ in their implementation and auxiliary space usage.

5. Algorithm in Detail

Algorithm 1: Create the binary tree

1. Start the program.
2. Define a node structure containing data, a left child pointer, and a right child pointer.
3. Create a new node by allocating memory and initializing its data.
4. Set the left and right child pointers of the new node to NULL.
5. Connect the nodes using their left and right pointers to construct the binary tree.

6. Assign the first node as the root.
7. Pass the root to the counting functions.

Algorithm 2: Count different types of nodes

1. Initialize the counters totalNodes, leafNodes, internalNodes, oneChildNodes, and twoChildNodes to zero.
2. Call a recursive function with the root node.
3. Check whether the current node is NULL. If it is NULL, return from the function.
4. If the current node is not NULL, increment the total node counter.
5. Check whether both child pointers are NULL. If so, increment the leaf node counter and display the node's value.
6. Check whether at least one child exists. If so, increment the internal node counter.
7. Check whether exactly one child exists. If so, increment the one-child node counter.
8. Check whether both children exist. If so, increment the two-child node counter.
9. Recursively process the left subtree.
10. Recursively process the right subtree.
11. Continue until all nodes have been visited.
12. Display all five counters.

Algorithm 3: Recursive counting method

1. Start the recursive counting function by passing the root node.
2. If the root is NULL, return zero.
3. Count the current node as one.
4. Recursively count the nodes in the left subtree.
5. Recursively count the nodes in the right subtree.
6. Add the current node count, left subtree count, and right subtree count.
7. Return the total number of nodes.

Formula: $N = 1 + N_L + N_R$, where N is the total nodes in the current tree, N_L the nodes in the left subtree and N_R the nodes in the right subtree. If the current tree is empty, its node count is zero.

Algorithm 4: Iterative counting method using a queue

1. Start the iterative counting function.
2. If the root is NULL, return zero.
3. Create an empty queue.
4. Insert the root node into the queue.
5. Initialize the total node counter to zero.
6. Repeat while the queue is not empty.
7. Remove the front node from the queue.
8. Increment the total node counter by one.
9. If the removed node has a left child, insert the left child into the queue.

10. If the removed node has a right child, insert the right child into the queue.

11. Continue until the queue becomes empty.

12. Return the total number of nodes.

This method uses level-order traversal, in which nodes are visited level by level.

6. Flowchart

The flowchart represents the modified binary tree node-counting program, including counting nodes with exactly two children, displaying leaf node values, and comparing recursive and iterative methods. (More flowcharts are given in the separate Flowchart document.)

Flowchart 1: Counting different types of nodes

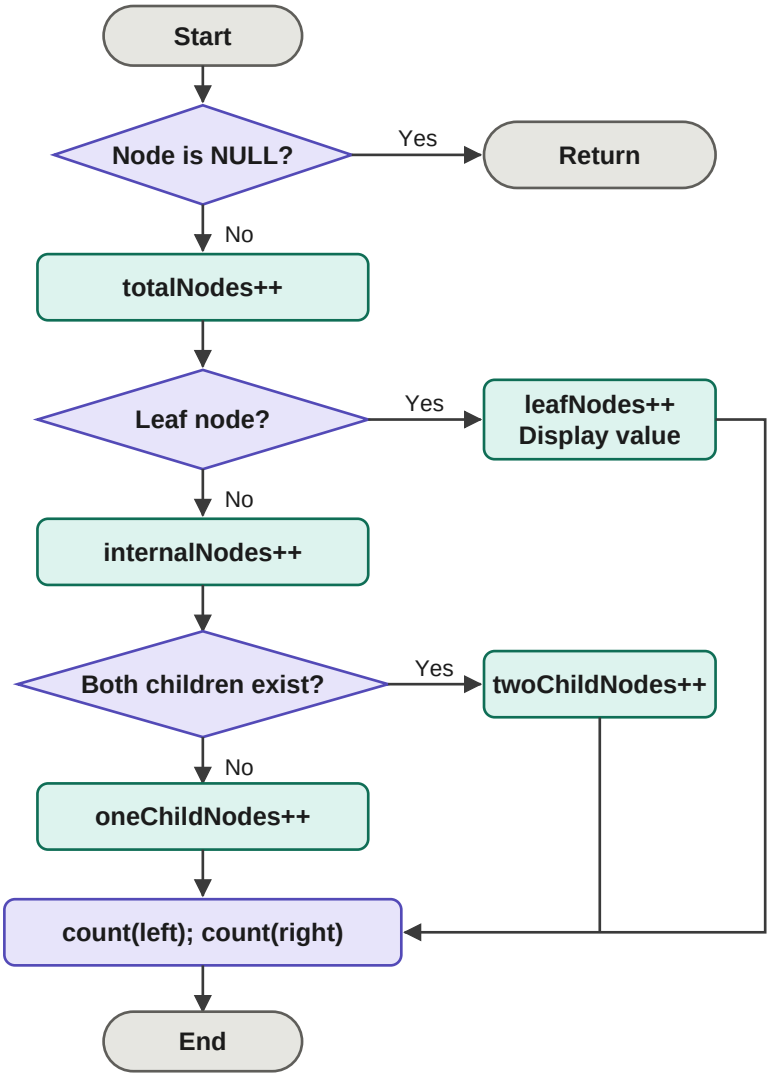

Figure 1: Recursive function that counts node types and displays leaf values

The traversal and counting decisions are performed for each non-NULL node. Recursive calls return when they reach NULL children. The final counts are displayed only after the initial traversal finishes.

Flowchart 2: Comparing recursive and iterative methods

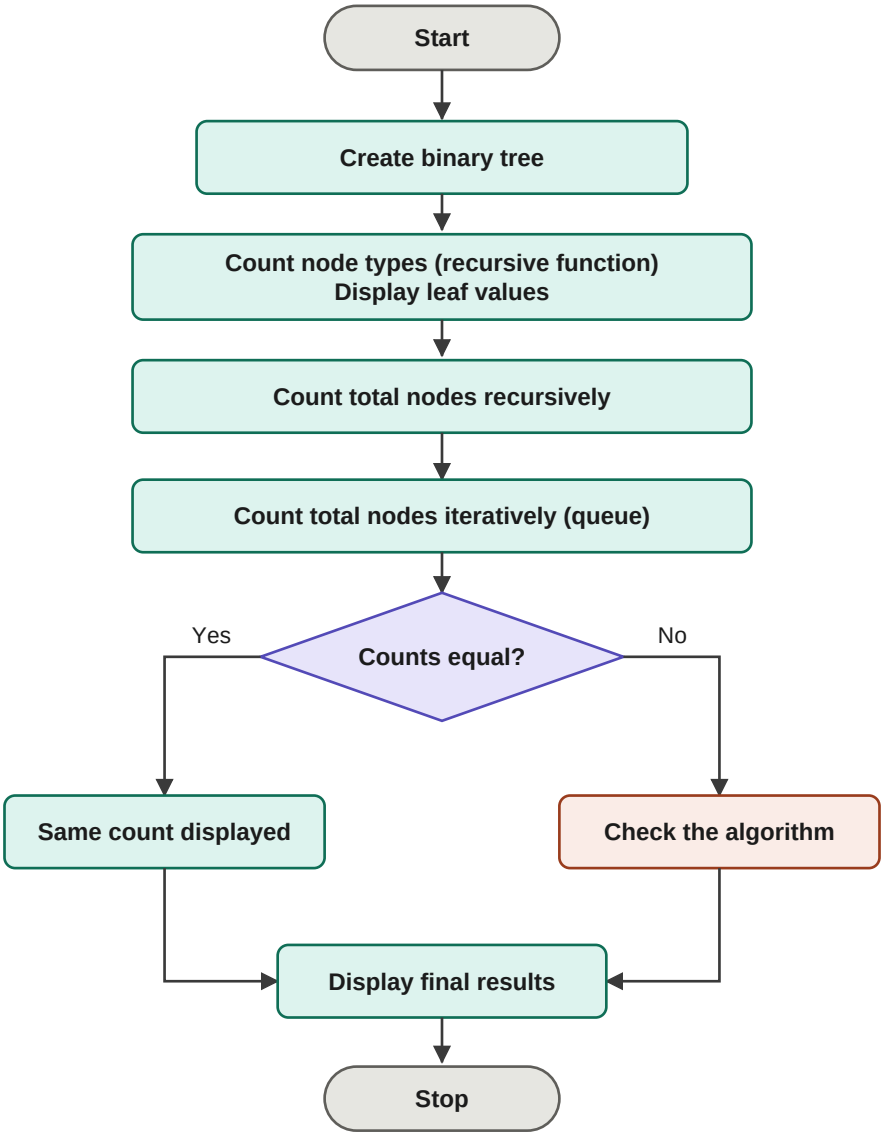

Figure 2: Main program comparing the recursive and iterative counts

7. Expected Output

For example, consider this binary tree:

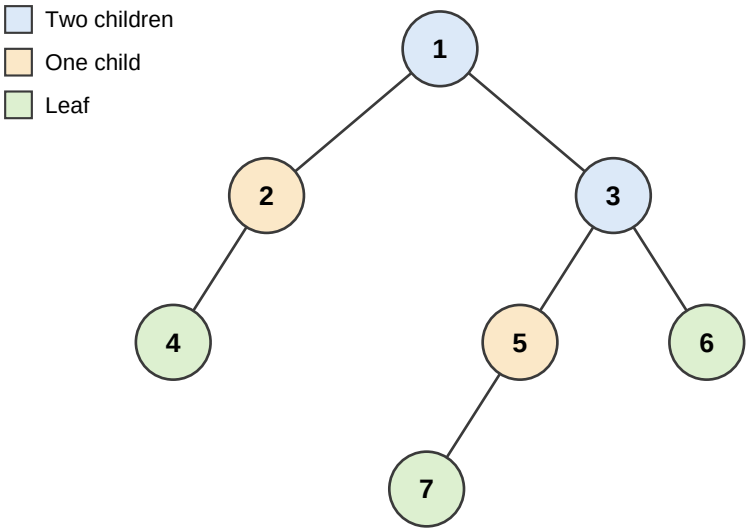

Figure 3: Example binary tree

The expected output is:

```
Leaf Nodes: 4 7 6
Total Nodes: 7
Leaf Nodes Count: 3
Internal Nodes: 4
Nodes Having Exactly One Child: 2
Nodes Having Exactly Two Children: 2

Recursive Node Count: 7
Iterative Node Count: 7

Both methods give the same result.
```

The leaf node display order depends on the traversal method. For preorder traversal, the leaf values are 4 7 6 (a level-order traversal would give 4 6 7). The tree contains seven nodes, three leaf nodes, four internal nodes, two nodes with exactly one child, and two nodes with exactly two children.

> This output was produced by compiling and running the program given in Section 9.

8. Comparison of Recursive and Iterative Methods

Feature	Recursive method	Iterative method (queue)	Iterative method (stack)
Technique	Function calls itself for left and right subtrees	Loop with a queue of nodes still to visit	Loop with a stack of nodes still to visit
Traversal order	Root, Left, Right (preorder style)	Level by level (level order)	Root, Left, Right (if right child is pushed first)
Stopping condition	Node is NULL (returns 0)	Queue becomes empty	Stack becomes empty
Time complexity	O(n)	O(n)	O(n)
Auxiliary space	O(h) call stack	O(w) queue, w = maximum width of the tree	O(h) explicit stack
Skewed tree (h = n)	O(n) stack depth	O(1): queue holds at most one node	O(1): stack holds at most one node
Perfect tree	O(log n)	About n/2 nodes, so O(n)	O(log n)

Feature	Recursive method	Iterative method (queue)	Iterative method (stack)
Stack overflow risk	Yes, for very deep trees	No (memory comes from the heap)	No (memory comes from the heap)
Code length	Shortest and simplest	Longer; needs queue header	Longer; needs stack header
Function calls / loop passes (n = 7)	15 calls (2n + 1)	7 loop passes	7 loop passes
Result for example tree	7	7	7

Conclusion of the comparison: both methods take O(n) time and give the same count. The recursive method is shorter and easier to read, but its stack depth equals the tree height, so it can overflow on very deep trees. The iterative queue method has no recursion limit, and its memory depends on the width of the tree instead of its height.

9. Program (C++ Source Code)

```
#include <iostream>
#include <queue>
#include <stack>
using namespace std;

// Node structure of the binary tree
struct Node {
    int data;
    Node *left;
    Node *right;
    Node(int value) : data(value), left(nullptr), right(nullptr) {}
};

// Recursive function: counts all node types and displays leaf values
void countTypes(Node* root, int &totalNodes, int &leafNodes,
                int &internalNodes, int &oneChildNodes, int &twoChildNodes) {
    if (root == nullptr)                    // base case
        return;

    totalNodes++;                           // every real node is counted

    if (root->left == nullptr && root->right == nullptr) {
        leafNodes++;                        // no children
        cout << root->data << " ";          // display the leaf value
    } else {
        internalNodes++;                    // at least one child
        if (root->left != nullptr && root->right != nullptr)
            twoChildNodes++;                // both children exist
        else
            oneChildNodes++;                // exactly one child
    }

    countTypes(root->left,  totalNodes, leafNodes, internalNodes, oneChildNodes, twoChildNodes);
    countTypes(root->right, totalNodes, leafNodes, internalNodes, oneChildNodes, twoChildNodes);
}

// Recursive total count:  N = 1 + N_left + N_right
int countRecursive(Node* root) {
    if (root == nullptr)
        return 0;
    return 1 + countRecursive(root->left) + countRecursive(root->right);
}

// Iterative total count using a queue (level-order traversal)
int countIterative(Node* root) {
    if (root == nullptr)
        return 0;
```

```
    queue<Node*> q;
    q.push(root);
    int count = 0;
    while (!q.empty()) {
        Node* current = q.front();
        q.pop();
        count++;
        if (current->left  != nullptr) q.push(current->left);
        if (current->right != nullptr) q.push(current->right);
    }
    return count;
}

// Iterative total count using a stack (alternative)
int countIterativeStack(Node* root) {
    if (root == nullptr)
        return 0;
    stack<Node*> st;
    st.push(root);
    int count = 0;
    while (!st.empty()) {
        Node* current = st.top();
        st.pop();
        count++;
        if (current->right != nullptr) st.push(current->right);
        if (current->left  != nullptr) st.push(current->left);
    }
    return count;
}

// Free the memory used by the tree (postorder)
void deleteTree(Node* root) {
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}

int main() {
    // Build the sample tree
    Node* root = new Node(1);
    root->left              = new Node(2);
    root->right             = new Node(3);
    root->left->left        = new Node(4);
    root->right->left       = new Node(5);
    root->right->right      = new Node(6);
    root->right->left->left = new Node(7);

    int totalNodes = 0, leafNodes = 0, internalNodes = 0;
    int oneChildNodes = 0, twoChildNodes = 0;

    cout << "Leaf Nodes: ";
    countTypes(root, totalNodes, leafNodes, internalNodes, oneChildNodes, twoChildNodes);
    cout << endl;

    cout << "Total Nodes: "                       << totalNodes     << endl;
    cout << "Leaf Nodes Count: "                  << leafNodes      << endl;
    cout << "Internal Nodes: "                    << internalNodes  << endl;
    cout << "Nodes Having Exactly One Child: "    << oneChildNodes  << endl;
    cout << "Nodes Having Exactly Two Children: " << twoChildNodes  << endl << endl;

    int rec = countRecursive(root);
    int itr = countIterative(root);
    cout << "Recursive Node Count: " << rec << endl;
    cout << "Iterative Node Count: " << itr << endl << endl;

    if (rec == itr && rec == countIterativeStack(root))
        cout << "Both methods give the same result." << endl;
    else
        cout << "Results are different - check the algorithm." << endl;
```

```
    deleteTree(root);
    return 0;
}
```

10. Explanation of the Code

Part of the code	Explanation
`struct Node`	Holds data and two child pointers. The constructor sets both pointers to nullptr so a new node starts as a leaf.
`countTypes(...)`	Recursive function; the five counters are passed by reference so that all calls update the same variables.
`if (root == nullptr) return;`	Base case that stops the recursion.
`totalNodes++`	Counts every real node.
`left == nullptr && right == nullptr`	Leaf test: increases leafNodes and prints the value with cout.
`else { internalNodes++; ... }`	At least one child exists. Inside it, if both children exist twoChildNodes++, otherwise oneChildNodes++.
`countRecursive(root)`	Returns 0 for NULL, otherwise 1 + count(left) + count(right).
`countIterative(root)`	Uses queue<Node*>: pop a node, count it, push its non-NULL children, repeat until the queue is empty.
`countIterativeStack(root)`	Same as the queue version but with a stack (right child pushed first so the left is processed first).
`deleteTree(root)`	Frees all nodes using a postorder traversal.
`main()`	Builds the example tree, prints the leaf values and counts, compares the recursive, queue and stack totals, and prints the result message.

11. Working of the Program

The recursive function starts at the root and visits each node in preorder (Root, Left, Right): 1, 2, 4, 3, 5, 7, 6. At each node it checks the two child pointers and updates the matching counters. Leaf values are printed at the moment they are found.

Step	Node	Type	total	leaf	internal	one	two	Leaves shown
1	1	Two children	1	0	1	0	1	(none)
2	2	One child	2	0	2	1	1	(none)
3	4	Leaf	3	1	2	1	1	4
4	3	Two children	4	1	3	1	2	4
5	5	One child	5	1	4	2	2	4
6	7	Leaf	6	2	4	2	2	4 7
7	6	Leaf	7	3	4	2	2	4 7 6

The iterative method visits the nodes level by level (1, 2, 3, 4, 5, 6, 7), keeping the waiting nodes in a queue, and reaches the same total of 7.

Node popped	Queue after pushing children	count
1	[2, 3]	1

Node popped	Queue after pushing children	count
2	[3, 4]	2
3	[4, 5, 6]	3
4	[5, 6]	4
5	[6, 7]	5
6	[7]	6
7	[ ] (empty)	7

12. Test Cases

Test	Tree	total	leaf	internal	one	two	Rec.	Iter.
1	Empty tree	0	0	0	0	0	0	0
2	Single node	1	1	0	0	0	1	1
3	Left-skewed, 4 nodes	4	1	3	3	0	4	4
4	Right-skewed, 4 nodes	4	1	3	3	0	4	4
5	Perfect tree, 7 nodes	7	4	3	0	3	7	7
6	Full tree, 5 nodes (root, two children, left child has two leaves)	5	3	2	0	2	5	5
7	Example tree of this document	7	3	4	2	2	7	7

In every case total = leaf + internal, internal = one + two, leaf = two + 1 (for non-empty trees), and the recursive and iterative counts are equal.

13. Time and Space Complexity

Method	Time	Auxiliary space	Notes
Counting node types (recursive)	O(n)	O(h)	One visit per node.
Recursive total count	O(n)	O(h)	2n + 1 calls; O(n) space for a skewed tree.
Iterative total count (queue)	O(n)	O(w)	About n/2 nodes in the queue for a perfect tree; O(1) for a skewed tree.
Iterative total count (stack)	O(n)	O(h)	Explicit stack in heap memory; no recursion limit.

Here n is the number of nodes, h the height and w the maximum width of the tree.

14. Advantages and Limitations

Advantages

- One traversal gives all five counts and the leaf values.
- Two independent counting methods allow the result to be cross-checked.
- The identities total = leaf + internal and leaf = two + 1 provide a quick self-check.
- Linear time for every method.

Limitations

- Deep recursion can overflow the stack on very tall trees.
- The queue method can use a large amount of memory for very wide trees.
- The sample program builds a fixed tree; reading a tree from the user needs extra code.
- The order in which leaf values are displayed depends on the traversal used.

15. Applications

- Checking whether a tree is full (no one-child nodes) or finding unbalanced parts.
- Listing terminal outcomes (leaves) of decision trees.
- Analysing expression trees and parse trees in compilers.
- Studying and comparing recursive and iterative algorithms and their memory use.
- Analysing hierarchical data such as folders and organisation charts.

16. Viva Questions and Answers

Q	Question	Answer
1	What new features does the modified program add?	Counting two-child nodes, displaying leaf values, and comparing recursive and iterative total counts.
2	What is a node with exactly two children?	A node whose left and right pointers are both non-NULL.
3	Why is a one-child node also an internal node?	Because it has at least one child; internal nodes are all non-leaf nodes.
4	What is the base case of the recursive count?	root == nullptr, which returns 0.
5	Write the recursive formula for the node count.	N = 1 + N_L + N_R.
6	Which traversal does the iterative queue method use?	Level-order traversal.
7	Why is a queue used?	It stores nodes still to be visited and gives them back in the order they were added.
8	Can a stack be used instead?	Yes. It gives a preorder (depth-first) visit and needs O(h) extra space.
9	What is the time complexity of both methods?	O(n).
10	How do their space needs differ?	Recursive: O(h) call stack. Queue: O(w). Stack: O(h).
11	When is the iterative queue method better?	For very deep or skewed trees where recursion may overflow the stack.
12	Why can the leaf order change?	Preorder visits 4, 7, 6; level order visits 4, 6, 7.
13	How do you verify the counts?	total = leaf + internal; internal = one + two; leaf = two + 1.

17. Conclusion

The modified program extends the original node-counting program by counting nodes with exactly two children, displaying the values of leaf nodes, and counting the total number of nodes with both a recursive and an iterative method. The comparison shows that both methods give the same result in O(n) time, while their auxiliary space differs: the recursive method depends on the tree height and the queue method on the tree width. This helps in choosing the right method for a given tree.

18. References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, MIT Press.
- Mark Allen Weiss, *Data Structures and Algorithm Analysis in C++*, Pearson.
- Ellis Horowitz, Sartaj Sahni, Dinesh Mehta, *Fundamentals of Data Structures in C++*.
- Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.